

强化学习 · RL Learning

Part 3 — CarRacing PPO 实战:从公式到能跑的代码

连续高斯策略 · GAE 的边界处理 · PPO Loss 的实现层陷阱 · 完整纠错记录

目录

- 0 本篇定位:这一份记录的是什么
- 1 项目结构与分工
- 2 环境管道:像素、帧栈、动作空间
- 3 第 1 题 · 连续高斯策略(act / ev)
- 4 第 2 题 · GAE:本次讨论的主战场
- 5 第 3 题 · PPO Loss:数学对了,实现还有六个坑
- 6 大数定律:蒙特卡洛为什么成立(全篇的地基)
- 7 骨架自带的两个 Bug(不是填空题的锅)
- 8 跑起来:算力、时间、加速路径
- 9 诊断实验清单
- 10 遗留问题与下一步
- 附录 A 完整可运行代码(三处填空 + 两处修复)

0 本篇定位:这一份记录的是什么

Part 1 讲 RLHF → PPO 的建模与求解,Part 2 是五个诊断案例。这一篇不同:它是把前两篇推过的公式,亲手写成能跑的代码的全过程记录,环境是 Gymnasium 的 `CarRacing-v3` ——96×96 像素输入、三维连续动作、俯视赛车。

之所以值得单独成篇,是因为「公式对」和「代码对」之间隔着一层东西。这一层不在任何论文里,只在实现细节中:数组的长度差一位、终止标志的语义、遍历方向、张量的 dtype、以及标量与向量的区别。这一轮实践里,数学部分几乎没有出错,而所有的错都出在这一层。所以本篇刻意保留了全部的错误版本和纠正过程,而不是只给最终答案——最终答案十几行,纠错过程才是这次真正学到的东西。

规则说明。整个过程遵循固定分工:我(用户)写,AI 只做正确性判定——不给提示、不给答案、不预测 bug、不反向推测思路。这个规则在 GAE 的最后一格上一度失效(连续四轮判错却没有推进),后文 § 4.9 如实记录了这次失效以及规则调整。

1 项目结构与分工

notebook 共 17 个 cell,分工如下:

模块	内容	谁写
环境管道	像素预处理、frame stack、动作变换、 <code>make_env</code>	AI(给定)
网络骨架	CNN backbone(Nature-DQN 配置)、flatten 维度自动推导、三个 head	AI(给定)
TODO 1	<code>act / ev</code> :连续高斯策略的采样与评估	我
TODO 2	<code>compute_gae</code> :优势函数的倒序递推	我
TODO 3	<code>ppo_update</code> :clip surrogate + value loss + entropy	我
训练主循环	采样 → GAE → 更新 → 录 GIF	AI(给定)

设计意图很明确:凡是**推导过的东西**都留空,凡是**纯工程脚手架**都给好。这样卡住的地方一定是理解问题,而不是环境装不上、维度算不对这类噪音。

2 环境管道:像素、帧栈、动作空间

2.1 像素预处理

```
def preprocess(f):
    g = np.dot(f[...,:3],[0.299,0.587,0.114])[6:90,6:90]
    return (g/255.).astype(np.float32)
```

三件事:RGB 转灰度(用的是 ITU-R BT.601 的亮度权重,绿色权重最大是因为人眼对绿最敏感)、裁掉边缘只留 84×84 (原图 96×96 底部是仪表盘,信息量低)、归一化到 $[0,1]$ 。 84×84 这个尺寸不是随意的,它是 Nature-DQN 的标准输入尺寸,后面的 CNN 配置直接对应。

2.2 frame stack:为什么是 4 帧

```
class FrameStack:
    def __init__(s,e,k=4): s.e=e; s.k=k; s.f=deque(maxlen=k)
```

核心理由是**单帧不构成马尔可夫状态**。给你一张赛车的静止照片,你无法判断它在往哪开、开多快、是不是在打滑——这些量是速度和加速度,必须从相邻帧的差分里读。MDP 的前提是「当前状态包含决策所需的全部信息」,单帧违反了这个前提,叠 4 帧后近似恢复。

为什么是 4 而不是 2 或 8:2 帧只够读速度,读不出加速度;8 帧的信息增益已经很小,但显存和计算翻倍。4 是 DQN 那篇论文的经验值,后来成了默认。

2.3 动作空间与 `to_env_action`

CarRacing 的动作是三维连续:方向盘 $[-1,1]$ 、油门 $[0,1]$ 、刹车 $[0,1]$ 。网络输出的是无约束的高斯采样值,需要变换到合法范围:

```
def to_env_action(a):
    return np.array([np.tanh(a[0]), np.clip(a[1],0,1), np.clip(a[2],0,1)], np.float32)
```

这里埋着一个理论上的近似,后面 § 10 会展开。存进 buffer 的是变换前的原始动作,所以 `act` 和 `ev` 算 `log-prob` 时用的是同一个量,前后自洽、不会算错。但这意味着策略实际上没有对 `tanh` 变换做 Jacobian 修正(标准做法是 `tanh-Gaussian` 加 $\log(1 - \tanh^2)$ 项)。这是一个有意的简化,不影响能不能训起来。

3 第 1 题 · 连续高斯策略(act / ev)

3.1 为什么是高斯,不是 softmax

离散动作空间可以对每个动作输出一个概率,softmax 归一化即可。连续动作空间有无穷多个动作,没法逐个列举,只能参数化一个分布。高斯是最自然的选择:两个参数(均值、标准差)、`log-density` 有闭式、采样便宜、支持重参数化。

网络的三个 head 对应: `mean_head` 给均值(依赖状态)、`log_std` 是一个与状态无关的可学习参数、`v_head` 给状态价值。让 $\log \sigma$ 独立于状态是常见做法——它承担全局探索强度的角色,训练过程中逐渐收缩。用 $\log \sigma$ 而不是 σ 是为了保证正性且优化尺度合理。

3.2 act 与 ev:一个曾经踩过的坑

```
def act(self, x):
    # 采样阶段(no_grad)
    mean, std, value = self.forward(x)
    dist = torch.distributions.Normal(mean, std)
    a = dist.sample()
    log_pro_total = dist.log_prob(a).sum(-1)
    return a, log_pro_total, value

def ev(self, x, a):
    # 更新阶段(要梯度)
    mean, std, value = self.forward(x)
    dist = torch.distributions.Normal(mean, std)
    log_pro_total = dist.log_prob(a).sum(-1)
    entropy = dist.entropy().sum(-1)
    return log_pro_total, entropy, value
```

两者的关键差别:`act` 产生动作,`ev` 评估一个已经产生的动作。此前的一版里 `ev` 内部写了 `a = dist.sample()`,把传进来的动作覆盖掉了——这是我自己抓出来的。后果很隐蔽:代码照跑不误,但 importance ratio ρ 变成了两个独立采样之间的比值,和 PPO 的语义完全脱钩,训练会缓慢地跑偏而不报任何错。

3.3 `sum(-1)` 的来源

三个动作维度用的是对角高斯(各维独立),联合密度是各维密度之积,取对数后变成求和:

$$\log \pi_{\theta}(a \mid s) = \sum_{i=1}^3 \log \mathcal{N}(a_i; \mu_i(s), \sigma_i)$$

所以 `.sum(-1)` 不是随手加的降维操作,它承载的是「各维独立」这个建模假设。entropy 同理。

4 第 2 题 · GAE:本次讨论的主战场

4.1 定义回顾

单步 TD 残差:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE 是不同步数优势估计的指数加权平均,写成 δ 的加权和:

$$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

等价的递推形式(这一条是写代码的直接依据):

$$A_t = \delta_t + \gamma \lambda A_{t+1}$$

两个衰减因子的分工要分清: γ 是 MDP 自带的折扣,决定「多远的未来还算数」; λ 是估计器的超参,决定「在偏差和方差之间怎么取舍」。 $\lambda = 0$ 退化成单步 TD(低方差高偏差,完全信任 V), $\lambda = 1$ 退化成蒙特卡洛(无偏但高方差,完全信任实际回报)。

4.2 done 到底是什么

函数签名是 `compute_gae(rews, vals, done, gamma, lam)`,长度关系是:

数组	长度	含义
<code>rews</code>	T	第 t 步得到的即时奖励
<code>done</code>	T	第 t 步执行完后 episode 是否结束
<code>vals</code>	T+1	$V(s_0) \dots V(s_{T-1})$ 再加一个末尾 bootstrap $V(s_T)$

`vals` 比另外两个多一位,这一位是主循环在 rollout 结束后额外前向一次拿到的:

```
with torch.no_grad():
    _, lv = model.act(torch.tensor(o, ...).unsqueeze(0))
    V.append(float(lv))
```

4.3 一个 rollout 里有多个 episode

这是理解 `done` 的关键一步。采样循环是**固定步数**而非固定局数:

```
for _ in range(n_steps):          # n_steps = 2000,不是一局
    ...
    D.append(float(t or tr))
    if t or tr: o, _ = e.reset()   # 局结束就地重置,继续填满 2000 步
```

所以一个 rollout 里通常包含**多个完整 episode**——CarRacing 早期策略经常几十步就撞出赛道,2000 步里塞十几局是常态。每一个 `done[t]=True` 都标记一次局的结束,末尾那个并不特殊:它可能为 True(恰好

在边界结束),也可能为 False(局还没打完就被 rollout 截断)。

一处被纠正的判断。我最初的说法是「如果 `done` 数组的最后一个为 True 就是游戏结束,不是 True 就是截断」——这句话把因果说反了。正确的是:截断的标志恰恰是最后一个 `done` 为 False(局还在进行中却被步数上限切断)。位置推不出性质,只有语义能。

4.4 terminated 与 truncated:一个真实存在的偏差源

Gymnasium 的新 API 把结束分成两种: `terminated` (环境自身的终止条件,比如撞车、任务完成)和 `truncated` (外部时间上限切断)。二者对应的 s_{t+1} 性质完全不同:

情况	s_{t+1} 的价值	正确处理
terminated	无未来可言, $V(s_{t+1}) = 0$	bootstrap 项归零
truncated	状态仍有价值,只是我们不看了	仍需 bootstrap,归零会引入偏差
rollout 边界(<code>done=False</code>)	同上	用 $V(s_T)$ bootstrap

而这份 notebook 的采样代码写的是:

```
D.append(float(t or tr))          # ← terminated 和 truncated 被合并了
```

同时第 7 个 markdown cell 的提示又要求「`done` 那步 $\gamma V(s_{t+1})$ 和递推都乘 $(1 - \text{done})$ 」。两边对不上:按提示写, `truncated` 也会被当成真终止、bootstrap 被错误归零。

这不是罕见路径。CarRacing-v3 的 TimeLimit 是 1000 步,而 `n_steps=2000`,意味着每个 rollout 至少撞上一到两次截断。每次都把一个本该有价值的状态估成零,value 会被系统性压低。而且它不报错、不 NaN、不崩——只是学得慢一点、歪一点。这正是「跑得通但学歪」的典型形态,也是最难在 code review 里看出来的一类问题。

本次练习的处理:按签名的假设(`done` 视为真终止)完成填空,先跑通;这个偏差留作后续修复项(见 § 10)。要修需要改签名——多传一个数组,或者传一个预先算好的 `next_value_mask`。

4.5 为什么必须在 rollout 层面算,不能在 mini-batch 层面

递推式 $A_t = \delta_t + \gamma \lambda A_{t+1}$ 依赖时序上的下一个元素。mini-batch 是打乱后随机抽的,一旦 shuffle, A_{t+1} 根本不在这一批里。所以顺序是刚性的:

```
采完整个 rollout(时序完整)
→ 算一次 GAE,adv / ret 固化成张量存进 buffer
→ 打乱、切 mini-batch、跑 epochs 轮
→ 每一批只是在取已经算好的数
```

这也解释了为什么 `ppo_update` 的入参里 `adv` 和 `ret` 是已经算好的常量而不是现算——它们在多个 epoch 之间保持不变,这正是 PPO「用同一批数据多次更新」的前提。

4.6 为什么倒序

先把两个量分开看:

- δ 没有方向问题。 δ_t 只用到 `rewards[t]`、`vals[t]`、`vals[t+1]`、`done[t]`,全是本地量,整个数组可以一次性向量化算出来,正序倒序都行。
- A 必须倒序。 A_t 依赖 A_{t+1} 。正序扫到 $t = 0$ 时, A_1 还没算,拿不到。

正序不是「不能算」,而是「代价不同」:正序要么先展开成 $\sum (\gamma\lambda)^l \delta_{t+l}$ 逐项累加(每个 t 都要往后扫一遍, $O(T^2)$),要么就得先倒序算一遍再正序用(等于绕了一圈)。倒序一次扫完是 $O(T)$ 。差的是复杂度,不是正确性。

4.7 两处切断,不是一处

当 `done[t]=True`,需要切断的是两个不同的东西:

1. δ_t 里的 bootstrap: $\gamma V(s_{t+1})$ 要去掉,因为 s_{t+1} 不属于这一局。此时 $\delta_t = r_t - V(s_t)$ 。
2. 递推的累积项: $\gamma\lambda A_{t+1}$ 要去掉,否则下一局的优势会顺着递推漏进上一局的末尾。此时 $A_t = \delta_t$ 。

骨架给的写法是统一乘 $(1 - \text{done}_t)$,我选择用分支表达。两者是同一件事的两种写法,不是两种做法——一度我说「和 $1 - \text{done}$ 无关」,这个说法把结论推过头了:不需要写成乘法是对的,说无关就错了。

4.8 末尾那一格: $A_{T-1} = \delta_{T-1}$

这一格是整个练习卡得最久的地方,值得完整展开。把定义式在 $t = T - 1$ 处展开:

$$A_{T-1} = \delta_{T-1} + (\gamma\lambda) \delta_T + (\gamma\lambda)^2 \delta_{T+1} + \dots$$

而 `delta` 数组是 `for t in range(T)` 填的,最大下标 $T - 1$ 。所以 δ_T 及之后根本不存在, $l \geq 1$ 的项全部消失:

$A_{T-1} = \delta_{T-1}$

如果非要给 A_T 定义一个值,那是 0——空和等于零。这正是骨架里 `last = 0.0` 的含义:它就是「 A_{t+1} 」这个滚动变量,初值设成 0,等价于声明 $A_T = 0$ 。有了它,末尾那一步走通用公式 $\delta_t + \gamma\lambda \cdot 0 = \delta_t$,特判自动消失。

末尾 bootstrap 已经用掉了,不要重复计入。rollout 边界的 $V(s_T)$ 是通过 `delta[T-1]` 里的 $\gamma \text{vals}[T]$ 进入计算的。在 `adv[T-1]` 上再加一次 `vals[-1]`,等于把同一个量算了两遍——而且量纲也不对: $\gamma\lambda \cdot X$ 这个结构里 X 的位置只能放 advantage,不能放 value。

4.9 四轮迭代的纠错记录

这一格连续写错四次,方向都是「往 `vals` 里找一个数补上去」。如实记录:

版本	写法	错在哪
v1	<code>adv[t] = vals[:-1]</code>	形状错。右边是长度 T 的数组,左边是标量位置。
v2	<code>adv[t] = vals[-1]</code>	形状对了,值错。 $V(s_T)$ 是 value 不是 advantage。
v3	<code>adv[t] = vals[T-1]</code>	同上,仍是在 <code>vals</code> 里挑下标——但 <code>vals</code> 里没有任何一个元素是 advantage。
v4	<code>adv[t] = delta[t] + $\gamma\lambda \cdot \text{vals}[-1]$</code>	结构对了一半,但 $\gamma\lambda$ 后面接的仍是 value;且 $V(s_T)$ 已在 <code>delta[T-1]</code> 中用过,重复计入。
正解	<code>adv[t] = delta[t]</code>	—

另有一处在 v1→v2 之间补上的:最初只判了「rollout 末尾」这一个特例,漏了「每一个 `dones[t]=True` 都要切断」。假如 $t = 500$ 那步结束了一局,原写法照样把 A_{501} 乘进来——那是下一局的优势。

关于协作规则的一次调整。四轮里 AI 每次都只回「错,回到定义」,结果是我在猜、它在判,没有信息增量。这是「只做正确性判定、不给提示」这条规则在**诊断信息不足**时的失效模式:判定本身是对的(四个版本都会导致训练异常),但重复同一句反馈四次并不构成有效的反馈回路。第五轮改为直接给出推导和答案后立刻解决。

结论:「不给答案」适用于我方向正确、只差临门一脚的场合;当连续两次尝试的方向系统性偏离时,应当切换为给出推导。这一条对以后所有的推导协作都适用。

4.10 最终实现

```
def compute_gae(rews, vals, dones, gamma=0.99, lam=0.95):
    T = len(rews); adv = np.zeros(T, np.float32)
    delta = []
    for t in range(T):
        #  $\delta$  可正序、可向量化
        if dones[t] == False:
            delta_t = rews[t] + gamma * vals[t+1] - vals[t]
        else:
            delta_t = rews[t] - vals[t]
        # 切断 ①:bootstrap
        delta.append(delta_t)

    for t in range(T-1, -1, -1):
        # A 必须倒序
        if t+1 == T:
            adv[t] = delta[t]
            #  $A_T = 0$  (空和)
        elif dones[t] == True:
            adv[t] = delta[t]
            # 切断 ②:递推累积
        else:
            adv[t] = delta[t] + gamma * lam * adv[t+1]

    return adv, adv + vals[:-1]
```

可选的收拢:用 `last` 滚动变量可以把三个分支压成两个、两个循环合成一个(倒序时当场算 δ_t ,不必先存 list)。功能等价,这里保留了展开形式以便对照定义。

4.11 第二个返回值的正确名字

返回的第二项 `adv + vals[:-1]` 是 value head 的回归目标。由 $A = Q - V$ 移项得 $Q = A + V$ 。

一处被纠正的表述。我(AI)最初把它写成「 $A = R - V$ 的移项,所以 $R = A + V$ 」——这是错的,移项得到的是 $Q(s_t, a_t)$ 而不是折扣回报 R_t 。

更精确地说:这个量是 λ -return。GAE 的 λ 加权和加回 V 之后,恰好等于 TD(λ) 的 return。 $\lambda = 1$ 时退化成蒙特卡洛回报, $\lambda = 0$ 时退化成单步 TD 目标 $r_t + \gamma V(s_{t+1})$ 。文献里叫它 "return" 不算错,但把它等同于普通折扣回报就错了。

它之所以能当 V 的回归目标,是因为 $V(s) = \mathbb{E}_a[Q(s, a)]$,而手上只有实际采到的那个 a ——所以这是 $V(s_t)$ 的一个单样本估计。

5 第 3 题 · PPO Loss:数学对了,实现还有六个坑

5.1 三项结构与符号

$$L = -\mathbb{E}[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)] + c_1 \text{VL} - c_2 \mathcal{H}$$

其中 $\rho_t = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}})$ 是重要性采样比。三项的符号各有来源:

- **policy 项取负**:目标是最大化 surrogate,而优化器最小化 loss。
- **value 项取正**:MSE 本身就是要最小化的。
- **entropy 项取负**:希望熵大(保持探索),所以减去它。 $c_2 = 0.01$ 是标准值,调成 0 就是 § 8 的探索诊断实验。

`clip` 夹的是 ρ 而不是 A :它约束的是「新旧策略偏离多远」,与优势的大小无关。这一点在 Part 1 里推过。

5.2 第一版的六个实现问题

数学结构和符号第一次就写对了,问题全在实现层。按「会不会炸」排序:

#	问题	后果
1	<code>A</code> 未定义(应为 <code>adv[b]</code>)	<code>NameError</code> ,立刻炸
2	<code>old_logp</code> 没切片	(N,) 对 (mb,) 广播成 (mb,N),不报错但全错
3	裸 <code>exp / min / np.mean</code>	内建函数语义不同; <code>np.mean</code> 作用于 tensor 会断梯度
4	<code>sum((v-ret)*(v-ret), axis=-1)</code>	是平方和不是均方,数值大 mb 倍(128×), c_1 失效
5	<code>ent</code> 未取均值	loss 是向量, <code>.backward()</code> 直接报错
6	policy 项符号写成 <code>+</code>	梯度方向整体反转,agent 会主动追求低优势动作,训练必崩

第 3 条值得单独说: `torch.min` 是逐元素取小,Python 内建 `min` 是比较两个对象的大小并返回其中一个。对张量调用后者,得到的是「整个张量 A」或「整个张量 B」,而不是逐元素的 `min`——语义完全不同,而且不一定报错。

第 6 条是在第二轮修改中新引入的:第一版写的 `loss = mean(...)` 本来是对的,因为我给的提示「别再多加一个负号」有歧义(本意是「不要在已经是负号的基础上再翻一次」),导致改成了 `+`。这是一次由反馈措辞不清造成的回退。

5.3 最终实现

```
def ppo_update(model, opt, obs, acts, old_logp, adv, ret,
               clip=0.2, c1=0.5, c2=0.01, epochs=4, mb=128):
    adv = (adv - adv.mean())/(adv.std()+1e-8)      # 整个 rollout 归一化一次
    N = len(obs)
    for _ in range(epochs):
        idx = torch.randperm(N, device=device)
        for s in range(0, N, mb):
            b = idx[s:s+mb]
            new_logp, ent, v = model.ev(obs[b], acts[b])

            loss = c1 * F.mse_loss(v, ret[b])
            loss = loss - c2 * torch.mean(ent)
            ratio = torch.exp(new_logp - old_logp[b])
            loss = loss - torch.mean(torch.min(
                ratio * adv[b],
                torch.clip(ratio, 1-clip, 1+clip) * adv[b]))

    opt.zero_grad(); loss.backward(); opt.step()
```

5.4 advantage 标准化:为什么推导里没有这一步

这个问题问得很准——它确实不在任何推导里,PPO 原论文正文也没有,是 OpenAI baselines 实现里加的工程 trick,后来被普遍沿用。两个部分要分开评价:

- **减均值**:有理论支撑。策略梯度对加一个与动作无关的基线是不变的, $\mathbb{E}[\nabla_{\theta} \log \pi \cdot b] = 0$,所以减均值不改变梯度期望,只降方差。这就是 baseline 那一套。
- **除标准差**:没有不变性保证,它确实改变了梯度——等价于给学习率乘了一个自适应因子 $1/\sigma$ 。目的是让梯度尺度不随 reward 的量纲和训练阶段漂移。CarRacing 的 reward 尺度和 CartPole 差一个数量级,不除的话同一组超参(`lr=3e-4`、`clip=0.2`)没法通用。

所以它是有偏的,只是偏的方向服务于「让 clip 阈值和学习率在不同环境下都工作」,实践收益大于代价。ICLR 2022 的 *What Matters in On-Policy RL* 系统评测过这一批 trick,advantage normalization 属于少数几个稳定正收益的。

一个实现分歧点。这份代码把归一化放在 epoch 循环外面——整个 rollout 归一化一次。有些实现(如 CleanRL 默认)放在 mini-batch 内部逐批归一化,那样每批的均值方差不同,等于引入额外噪声,且小 batch 时估计不准。两种写法都有人用,值得知道差别在哪。

注意 `ret` 不做标准化:它是 value head 的回归目标,标准化后就和 `v` 不在一个尺度上了。

6 大数定律:蒙特卡洛为什么成立(全篇的地基)

前面所有的代码——GAE、PPO loss、advantage 标准化——都建立在同一件事上:用有限个样本的算术平均,去代替一个算不出来的期望。这一步之所以合法,靠的是大数定律。它不是某一处的技巧,而是整条路线的地基;把它抽掉,前面每一行 `.mean()` 都失去意义。

6.1 问题:期望本身是算不出来的

策略梯度的目标是

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{T \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \right]$$

这个期望要对所有可能的轨迹求和(或积分)。CarRacing 的动作是三维连续、一局上千步,轨迹空间的维度是天文数字,而且转移概率 $p(s' | s, a)$ 由物理引擎隐式给出,我们根本写不出它的解析形式。也就是说:这个期望既不可枚举,也不可积分。

唯一可行的出路是:既然我们没法算 $\mathbb{E}$,但我们能从这个分布里采样(让 agent 去跑就是采样),那就用样本平均近似它。这就是蒙特卡洛方法。而「样本平均能替代期望」这句话的证明,就是大数定律。

6.2 弱大数定律与它的证明

设 $X_1, \dots, X_n$ 独立同分布, $\mathbb{E}[X_i] = \mu, \text{Var}(X_i) = \sigma^2 < \infty$, 样本均值 $\bar{X}_n = \frac{1}{n} \sum X_i$ 。则对任意 $\varepsilon > 0$:

$$\lim_{n \rightarrow \infty} \mathbb{P}(|\bar{X}_n - \mu| \geq \varepsilon) = 0$$

证明只要两步,用切比雪夫不等式(这一条在数学线里单独推过):

第一步,算样本均值的方差。由独立性,方差可加:

$$\text{Var}(\bar{X}_n) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n}$$

第二步,代入切比雪夫 $\mathbb{P}(|X - \mu| \geq \varepsilon) \leq \text{Var}(X)/\varepsilon^2$:

$$\mathbb{P}(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{n \varepsilon^2} \xrightarrow{n \rightarrow \infty} 0$$

注意这个界的形状,它决定了后面一切工程手段的方向。偏离概率的上界是 $\sigma^2/(n\varepsilon^2)$ ——分子是方差,分母是样本量。要让估计更可靠,只有两条路:加大 n (采更多样本,贵)或减小 σ^2 (降方差,免费)。

PPO 里那一整套东西——baseline、GAE 的 λ 、advantage 标准化、多环境并行——全部落在这两条路上。这不是巧合,是这个不等式直接推出来的工程议程。

6.3 这份代码里,大数定律被调用了几次

把前面写的每一处 `.mean()` 摊开看,每一个都是一次「用样本平均代替期望」:

代码位置	被替代的期望	样本量 n
<code>torch.mean(torch.min(...))</code>	$\mathbb{E}_{s,a \sim \pi_{\text{old}}}[\min(\rho A, \text{clip} \cdot A)]$	mb = 128
<code>F.mse_loss(v, ret[b])</code>	$\mathbb{E}[(V_{\theta}(s) - \text{target})^2]$	mb = 128
<code>torch.mean(ent)</code>	$\mathbb{E}_s[\mathcal{H}(\pi(\cdot s))]$	mb = 128
<code>adv.mean() / adv.std()</code>	优势的一阶、二阶矩	N = 2000
<code>adv + vals[:-1]</code> (λ -return)	$V(s_t) = \mathbb{E}_a[Q(s_t, a)]$	n = 1

最后一行值得停一下:value head 的回归目标是**单样本估计**——我们只采到了一个动作,就拿它的 λ -return 当作 $V(s_t)$ 的估计值。 $n = 1$ 时大数定律给不出任何保证,单个样本的方差就是 σ^2 本身。

它之所以还能工作,靠的是另一重平均:同一个状态(或相似状态)会在训练过程中被反复访问,神经网络的拟合本身就在跨样本做平滑。换句话说,期望不是在单个 target 里取的,是在整个训练过程中通过梯度下降逐步取的。这也解释了为什么 value loss 天然比 policy loss 噪声大、为什么 $c_1 = 0.5$ 要压一压它的权重。

6.4 收敛「速度」由中心极限定理给

大数定律只说「 $n \rightarrow \infty$ 时会收敛」,不说收敛得多快。速度由 CLT 给:

$$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2) \implies \bar{X}_n - \mu \sim \frac{\sigma}{\sqrt{n}}$$

误差按 $1/\sqrt{n}$ 缩小——这个速率很慢:精度提高 10 倍,要多采 100 倍的样本。这正是 RL 样本效率差的数学根源,也是为什么在采样量固定的前提下,降方差(减小 σ)几乎是唯一划算的手段。

把 § 5.4 的两个操作重新放回这个框架里看,它们的地位就分明了:

- **减均值(baseline):** $\mathbb{E}[\nabla_{\theta} \log \pi \cdot b] = 0$, 所以 μ 不变、 σ 变小——严格地沿着「减小 σ^2 」这条路走,不付任何偏差代价。这是唯一免费的午餐。
- **除标准差:** 改变了 μ 的尺度,不属于这个框架内的合法操作,是纯工程的自适应学习率。
- **GAE 的 λ :** 直接就是在 σ^2 (方差)和偏差之间画一条连续的滑轨, $\lambda = 1$ 无偏高方差、 $\lambda = 0$ 有偏低方差。

6.5 CLT 的第二重身份:为什么假设高斯是合理的

连续策略用高斯,除了「参数少、密度有闭式」这些工程理由,还有一层统计上的支撑:回报是**多步奖励的和**, $R_t = \sum_l \gamma^l r_{t+l}$ 。多个随机量求和,在弱相关的条件下由 CLT 趋向正态。所以把与回报相关的量建模成高斯,不是纯粹的方便,而是有渐近依据的。

顺着这条线还能解释 value loss 为什么用 MSE:若假设 $\text{target} \sim \mathcal{N}(V_{\theta}(s), \sigma^2)$, 写出负对数似然

$$-\log p = \frac{(\text{target} - V_{\theta}(s))^2}{2\sigma^2} + \log \sigma + \frac{1}{2} \log 2\pi$$

对 θ 求导时后两项是常数,剩下的**就是 MSE**。所以 MSE 不是随手选的距离函数,它是「高斯噪声假设下的极大似然」。同理,离散策略的交叉熵是类别分布假设下的 MLE——这两件事是同一个模子里出来的。

6.6 大数定律的前提被破坏时会发生什么

定理有前提:独立同分布,且样本来自目标分布本身。RL 里这两条都很脆弱,而 PPO 的几个设计恰好是在修补它们:

被破坏的前提	在 RL 中的表现	对应的修补
样本来自目标分布	更新几步之后 π_θ 已经不是采样时的 π_{old} ,样本「过期」	重要性采样比 $\rho = \pi_\theta / \pi_{\text{old}}$ 把分布换算回来
换算后的估计要稳定	ρ 偏离 1 太远时方差爆炸(比值型估计量的经典病)	clip 把 ρ 夹在 $[1 - \epsilon, 1 + \epsilon]$;且 epochs 只跑 4 轮就重新采样
独立性	同一条轨迹内相邻样本高度相关,有效样本量远小于 N	多环境并行采样;mini-batch 前 shuffle

换个角度说:PPO 相对于朴素策略梯度多出来的每一个部件,都是在维护蒙特卡洛估计的有效性——要么保证它无偏(重要性采样),要么控制它的方差(clip、baseline、GAE),要么恢复独立性(并行 + shuffle)。理解了这一点,PPO 就不再是一堆需要背的公式,而是一条从「期望算不出来」出发的必然推演。

7 骨架自带的两个 Bug(不是填空题的锅)

7.1 terminated / truncated 合并

见 § 4.4。采样处 `D.append(float(t or tr))` 与提示文档要求的语义不一致,是骨架自身的缺陷。填空题按签名假设完成即可,修复留作后续。

7.2 dtype:Double vs Float

三处填空全部正确后,第一次运行仍然报错:

```
RuntimeError: Found dtype Double but expected Float
```

追溯: `adv` 是 `np.zeros(T, np.float32)`, `float32`; 而 `ret = adv + vals[:-1]`, 其中 `vals = np.array(V)` 由 Python float 列表构造,默认 `float64`。`float32 + float64` 广播成 `float64`,于是 `rt` 是 `Double`、`v` 是 `Float`,`mse_loss` 反传时类型冲突。

主循环里 `0t` / `At` / `Lpt` 三行都显式写了 `dtype=torch.float32`,唯独这一行漏了:

```
# 修复前
at=torch.tensor(adv,device=device); rt=torch.tensor(ret,device=device)
# 修复后
at=torch.tensor(adv,dtype=torch.float32,device=device)
rt=torch.tensor(ret,dtype=torch.float32,device=device)
```

这类 bug 的特征:错误信息离病因很远(报在 `backward()` 的 C++ engine 里,病因在三十行外的一次 numpy 广播)。排查方法是从报错的算子往回追每个入参的 dtype,而不是盯着报错行看。

8 跑起来:算力、时间、加速路径

8.1 资源核算

Colab Pro + T4(15 GB 显存):

- **显存:**完全不紧张。CNN 是 Nature-DQN 小配置,mini-batch $128 \times 4 \times 84 \times 84$,加上一个 rollout 的 2000 帧 obs($2000 \times 4 \times 84 \times 84 \text{ float32} \approx 226 \text{ MB}$),余量很大。
- **时间:** `iterations=300`, `n_steps=2000` 即 60 万环境步,T4 上约 3-6 小时。
- **瓶颈不在 GPU**,在 Box2D 的物理仿真——单环境串行采样时 GPU 大部分时间空转。

8.2 加速路径(按性价比)

手段	预期收益	代价
并行环境(<code>AsyncVectorEnv</code> ,8-16 个)	5-8×	<code>compute_gae</code> 要改成 (T, num_envs) 的并行版本,每个环境独立处理 done 切断
Frame skip (同动作重复 4 帧、reward 累加)	2-4×	十行以内,CarRacing 上公认正收益
降 <code>n_steps</code> 提 <code>iterations</code>	不减总时间	更新更频繁,早期进步可见得更早
AMP 混合精度	有限	这个 CNN 太小,不值得加复杂度

8.3 首次跑通的输出

```
[训练前 乱撞] 得分: -14.5
iter   0 | reward/ep ≈ -59.8
完成!
```

三处填空 + dtype 修复后链路打通,无 NaN、无报错。这两个数不构成任何趋势信号:第一个是随机策略跑 400 步的分,第二个是 iter 0 的统计,3 轮太短。

日志本身也有两处要注意。其一, `if it%10==0` 决定只有 iter 0/10/20... 才打印,iter 1、2 不是没跑而是没打日志。其二, `reward/ep = sum(R)/max(1,sum(D))` 的分母因为 D 混入了 truncated 而偏大,数值会系统性偏低——看趋势可以,当绝对值不行。

9 诊断实验清单

代码跑通之后,真正的学习在这里。每一项都对应 Part 2 里推过的一个诊断维度:

实验	改动	预期观察
探索坍塌	<code>c2</code> $\rightarrow$ 0	熵不再被鼓励,策略过早收敛到某个次优模式(比如原地打转),卡住不动
GAE 偏差/方差	<code>lam</code> $\rightarrow$ 0 / 1	0 时完全信任 V ,早期 V 不准会带偏;1 时无偏但方差大,曲线抖
Reward misspec	包一层只奖速度、不管在不在赛道	学出「冲出赛道飙车」——reward hacking 的最小可复现样本
泛化	CarRacing 每次 reset 换赛道	天然的泛化测试,不需要额外构造
截断偏差	修复 <code>t</code> or <code>tr</code> 前后对比	value loss 曲线和最终得分的差异——验证 § 4.4 的判断

第三项尤其值得做:它把 Part 1 里「reward misspec」这个抽象概念变成十行代码就能复现的现象,是面试里可以直接讲的素材。

10 遗留问题与下一步

1. **terminated / truncated 分离**。改采样端只存 terminated(省事),或改签名多收一个数组(更正确)。修完跑 A/B 对比,这本身就是一个完整的小实验。
2. **tanh 变换的 Jacobian 修正**。当前策略在无约束空间上定义分布、在外部做 squash,没有加 $\log(1 - \tanh^2 a)$ 的 log-det 项。标准做法是 tanh-Gaussian(SAC 里的写法)。不影响能否训起来,但动作分布的行为会和理论不一致。
3. **并行环境版 GAE**。这是从「会写单条轨迹的 GAE」到「会写生产级 GAE」的一步,数组变成二维、切断逻辑逐环境独立。
4. **观测归一化 / reward 缩放**。当前 obs 只除了 255,reward 原样使用。running mean/std 归一化是另一批常见 trick,可以作为对照实验。

附录 A 完整可运行代码

A.1 网络(TODO 1 已填)

```

class ActorCritic(nn.Module):
    def __init__(self, act_dim=3):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(4,32,8,4), nn.ReLU(),
            nn.Conv2d(32,64,4,2), nn.ReLU(),
            nn.Conv2d(64,64,3,1), nn.ReLU(), nn.Flatten())
        with torch.no_grad():
            n = self.conv(torch.zeros(1,4,84,84)).shape[1]
        self.fc = nn.Linear(n, 512)
        self.mean_head = nn.Linear(512, act_dim)
        self.log_std = nn.Parameter(torch.zeros(act_dim))
        self.v_head = nn.Linear(512, 1)

    def forward(self, x):
        h = F.relu(self.fc(self.conv(x)))
        mean = self.mean_head(h)
        std = self.log_std.exp().expand_as(mean)
        value = self.v_head(h).squeeze(-1)
        return mean, std, value

    def act(self, x):
        mean, std, value = self.forward(x)
        dist = torch.distributions.Normal(mean, std)
        a = dist.sample()
        return a, dist.log_prob(a).sum(-1), value

    def ev(self, x, a):
        mean, std, value = self.forward(x)
        dist = torch.distributions.Normal(mean, std)
        return dist.log_prob(a).sum(-1), dist.entropy().sum(-1), value

```

A.2 GAE(TODO 2)

```

def compute_gae(rews, vals, dones, gamma=0.99, lam=0.95):
    T = len(rews); adv = np.zeros(T, np.float32)
    delta = []
    for t in range(T):
        if dones[t] == False:
            delta_t = rews[t] + gamma * vals[t+1] - vals[t]
        else:
            delta_t = rews[t] - vals[t]
        delta.append(delta_t)
    for t in range(T-1, -1, -1):
        if t+1 == T:
            adv[t] = delta[t]
        elif dones[t] == True:
            adv[t] = delta[t]
        else:
            adv[t] = delta[t] + gamma * lam * adv[t+1]
    return adv, adv + vals[:-1]

```

A.3 PPO 更新(TODO 3)

```
def ppo_update(model, opt, obs, acts, old_logp, adv, ret,
               clip=0.2, c1=0.5, c2=0.01, epochs=4, mb=128):
    adv = (adv - adv.mean())/(adv.std()+1e-8)
    N = len(obs)
    for _ in range(epochs):
        idx = torch.randperm(N, device=device)
        for s in range(0, N, mb):
            b = idx[s:s+mb]
            new_logp, ent, v = model.ev(obs[b], acts[b])
            loss = c1 * F.mse_loss(v, ret[b])
            loss = loss - c2 * torch.mean(ent)
            ratio = torch.exp(new_logp - old_logp[b])
            loss = loss - torch.mean(torch.min(
                ratio * adv[b],
                torch.clip(ratio, 1-clip, 1+clip) * adv[b]))
            opt.zero_grad(); loss.backward(); opt.step()
```

A.4 主循环的 dtype 修复

```
at = torch.tensor(adv, dtype=torch.float32, device=device)
rt = torch.tensor(ret, dtype=torch.float32, device=device)
```

本篇是对话整理稿。主线推导与纠错记录均经双向核对,但正式引用前建议对 GAE 边界条件与 λ -return 的等价性自行再推一遍。文中标注为「被纠正」的条目,记录的是双方各自出错并被对方指出的实际过程。